SAVITRIBAI PHULE PUNE UNIVERSITY

MDM-271-ETC-OOP

Sem:4 7022414

Seat No 65003604
Stk. No 4808267
Sub. OOP
Centre 4036

...ation	SE 2025 Patter... May Jun 2...
...ate	29/Jun/2026
...ct	object oriente... Programing
...er No.	6625-257 Sec
...edium Answer	English

Seat No. : In figure & in word...

...	5	0	0	3	3	0	4	8

- Five - zero - zero...
...reet six - zero...
...ue) - Eight - zero...

...nature of ...ndidate	
...ture of	

Instruction to Candidate

1. Candidate has to confirm seat number, subject and centre number printed on Bar code and Write it on attendance sheet.
   विद्यार्थ्यांनी प्रथम बार कोडवरील आसन क्रमांक, विषय व केंद्र क्रमांक तपासून योग्य असल्याची खात्री करावी आणि उपस्थिती पत्रकावर नोंदवावी.
2. Paste Bar Code in prescribed space.
   उत्तरपत्रिकेवरील विहित जागेतच बार कोड लावावा.
3. Do not write anything on Bar code sticker, otherwise it will be treated as unfairmeans.
   बार कोड स्टिकरवर काहीही लिहू नये, अन्यथा परीक्षा गैरप्रकार समजला जाईल.

Q. No.	Examiner	Moderator
1	—	
2	07	
3	—	
4	11	
5	—	
6	07	
7	—	
8	04	
9	08	
10		
11		
12		
Total in Figure	037	
Total in Words	Three seven	
Signature		

Total	Marks in Figure	Marks in Words	Sign
Examiner	37	Three seven	
Moderator			

१. विद्यार्थ्याने उत्तरपत्रिकेच्या मुखपृष्ठावर तसेच उपस्थिती पत्रकावर विहित जागेत आसन क्रमांक अंकात व अक्षरात विनचूक लिहून स्वाक्षरी करावी.

Candidate shall fill all information about seat number, paper etc. in prescribed space and sign on the answer book and attendance sheet.

२. उत्तरपत्रिकेवर फक्त निळ्या अथवा काळ्या शाईचा उपयोग करावा, अन्यथा उत्तरपत्रिकेचे मूल्यमापन केले जाणार नाही.

Candidate shall use blue or black ink only. Otherwise answer book will not be evaluated.

३. उत्तरपत्रिकेल्या पृष्ठक्रमांक ३ पासून लिहिण्यास प्रारंभ करावा.

Candidate shall start writing answer from page no. 3 of the answer book.

४. संबंधित प्रश्नाचे अथवा उपप्रश्नाचे उत्तर जेथून सुरू होते तेथेच समासात प्रश्न क्रमांक, उपप्रश्न क्रमांक अचूक व स्पष्ट लिहावा, यासाठी वेगळ्या शाईचा उपयोग करू नये.

Candidate shall mention question number, sub question number correctly at the beginning of the same and shall not use ink other than blue or black.

५. प्रत्येक पानाच्या दोन्ही बाजूस लिहावे, उत्तरपत्रिका किंवा पुरवणी उत्तरपत्रिकेचे कोणतेही पान फाडू नये, फाडल्यास परीक्षा गैरप्रकार समजून पुढील कार्यवाही करण्यात येईल.

Candidate shall write on both sides of pages and shall not tear off any page, it will be treated as unfairmeans.

६. पेपर संपण्यापूर्वी १० मिनिटे अगोदर इशारा घंटा होईल, त्यानंतर विद्यार्थ्याने उत्तरपत्रिका व पुरवणी उत्तरपत्रिकेवर होलोक्राफ्ट स्टिकर विहित जागेवरच लावावा.

Warning bell will be given before 10 minutes of the concluding time. Candidate shall paste Holocraft Sticker at appropriate space on the answer book.

७. कॉपी करणे किंवा दुसऱ्याच्या नावावर परीक्षेस बसणे यांसारख्या कृती 'महाराष्ट्र-प्रीव्हेन्शन ऑफ मालप्रॅक्टिस ॲक्ट युनिव्हर्सिटी, बोर्ड ॲण्ड अदर स्पेसिफाईड एक्झामिनेशन्स ॲक्ट, १९८२' (सा.फु.पु.वि. चा अध्यादेश क्रमांक ९) त्यानुसार संमत केलेला कायदा या अन्वये दंडही असेल.

An Act of Copying or Impersonations at an Examination is Punishable under 'The Maharashtra Prevention of Malpractices at University, Board and Other Specified Examinations Act, 1982' (Ordinance 9 of SPPU). The Act passed to the effect.

Examiner and Moderator has to write marks on all given appropriate place only. Examiner should give assessment tick(✓) or (×) in the margin.

Q. No.	Examiner	Moderator	Verification	Revaluation
1	—			
2	07			
3	—			
4	11			
5	—			
6	07			
7	—			
8	04			
9	03			
10				
11				
12				
Total	87			

Q. No. / Q.No.

Q2]

a] object oriented programing :- it is the type of progrming In which classes, & objects are used. by Combining the properties, & Method in the class & creating a real instance of that class.

Features of OOPs

(1) Reusability :- With the help of extend keyword we can use the functions that are already written in the parent Class.

(2) Data Hiding :- can Hide the important imple mentation & data by using the abstract.

(3) Data Security :- Can Secure Methods & properties of one class from other files, modules & other classes. using Modules like privat.

(4) Flexibility :- Can Make changes of already existing functions & Methods. by overriding it in the child class.

(5) Modularity :- Can Make the Complex program into simple & understandable with lambda expression.

Q. No./Q.No.

Q.7

b →

- Member Function :- It is the Normal Method . or Function Where the proceder of execution is

main() → call() → Function implementation

- Mean the Call will goes from the main then the Function will be implemented

→ First is function declaration, definition then Calling

ex :- #include <iostream>
using Namespace std;
class A {
public:
int square(int x)
{
return x*x;
}
};
int main() {

class A s;
int result = s.square(4);
cout << " the square = " << result << endl;
}
// Output =

Q. No.

Q2

b]

Inline function:- In this method the definition of the function is directly written at the place of calling.

main() → obj → definition.

ex:-
```
#include<iostream>
using namespace std;
class A{
public:
    int square(int);
};

int main()
{
    A s;
    s() → {
        return x*x;
    }
int result = s.square(2);
    cout << "the Square =" << result << endl;
```

Output = 4

T. W./Q.No.

Q47

Q] → this keyword :- this keyword is the special type of keyword that refers to the current object of class.

- Whenever you create any object at that time the Memory allocation is done. & the Reference goes to the class of that object through that Reference the this keyword refers to current executing object

```
class Student
{
  int RollNo;
  String Name;

      Student(int R, String n)
      {
       this.RollNo = R;
       this.Name = n;
      }
} public class Main{
public static void main(String args[])
  {
    Student S = new Student();
    S.RollNo = 404;
    S.Name = "Akash";
  }
```

Y. K. /Q.No.

Q9

Q] →

```
System.out.println(S.RollNo);
System.out.println(S.Name);
    }
}
```

O/P = 404
= Akash.

- there is Student class having properties Name, & Roll No of Student

- & Constructor of the Student class.

- Studen S = new Student();
created an Refeerence variable of the Constructor student();

- A. S.RollNo = 404
Assign the value 404 to the property of Constructor

- S.Name = "Akash";
Assign the Name Akash to the ~~wanted~~ String Name of the Student class

Oh

Q. P. /Q.No.

Q 17

b]

```
public class Student {

    int RollNo;
    String Name;
    int marks;

i] -> Student (int R, int M, String N)
      {
      this.RollNo = R;
      this.Name = N;
      this.marks = M;
      }

ii] -> public String Grade (int M)
      {
        if (M > 50) {
          return "B" }

        if (M < 50) {
          return "C";
          }

        if (M >= 90) {
          return "A";
          }
      }
```

Q. No. / Q.No.

Q4]

b] → ~~public void Display (int R, int M, String N)~~

~~{~~

~~System.out.println(" ");~~

iii] → public void Display (int R, int M, String N)

{

System.out.println ("Name of Studen" + N);

System.out.println ("Roll no of Student" + R);

String Marks_of_S = Grade (int M);

System.out.println ("Grade of Student" + Marks_of_S)

}

public class Main {

public static void main (String args[])

{

Student S = new Student (5, 99, Akash);

S.Display ();

}

// output

Name of Student Akash

Roll no of Student 5

Grade of Student A

	Q.No.						TOTAL
E							
M							

Q.No.	Method overloading	overriding.
(1)	Compile Time polymorphism	(1) Run Time polymorphism.
(2)	using same Function Name	(2) using Different function Name.
(3)	does Not uses special type of keyword	(3) uses special type of keyword i] extends.
(5)	Can be peeform if parameters, type of parameter, & order of parameter is different.	(5) Can be b peeform if child class extends the properties of parent class.
(6)	Method identified at the Compilation time	(6) Method identified at the Execution time.

P. T. /Q.No.

Q6

b]

```
class Parent {

    public void eat()
    {
        System.out.print("the Animal
                          is eating");
    }
    String Name = "Cat";
}

class Child extends Parent {

    public void sound()
    {
        System.out.print("the Animal
                          is listening
                          sound");
        System.out.println(" is Name);
    }

} public class Main {
public static void main(String args[]) {
    {
        Child c1 = new child;
        c1.sound();
    }
}
```

Q8

b] → O/P The Animal is listening Sound Cat.

- In this program, there is parent class Named → Parent
- And the class child extends the properties of Parent class
- property [one] → the animal is eating.
- property 2] → Name = Cat

and the child class have property 1] → the Animal is listning Sound.

- in the Main Method I have created the object of child class
- & called the Sound function
- & printed the property 2 of parent class

Q. No./Q.No.

Q8

Q] → packages :- ~~part~~ in the java programming there is Classes, Methods. In Any Application there are lot of classes & Method Init the program that creates Conflict, to overcom that Confusion java uses packages.

- It is just like Folder of Related Files.

ex:- Student.java
Teacher.java
book.java
library.java.
college.java.

College
- Student.java
- teacher.java
- college.java

library
- books.java
- library.java.

~~Folder~~ → package 1 = College
package 2 = library

Q.8

a → Benifits

1] Reduced the conflict & confusion of ~~source~~ identification of source code.

2] Redable
— Make simple to Read & understand to the programer.

3] easy to Debugg the application
— Debugging Makes simple.

4] error, exception handling
→ Make exception Handling simple to each programmer For each ~~Ent~~ Package.

b → lamda function / expression it way to evaluate the inheritance of the function with only single abstract Method.

— it is applicable For the Functional interFace only

Q. No./Q.No.

Q8

b→ Without lambda expression

ex:-

@FunctionalInterface
interface class_Nam()
{
public void sound();
}

public void child implements classNa
{

public void sound()
{
System.out.prin(" Animal
Sound is very
-louded !!);
}

public class Main {
public static void main (String args[])
Child c1 = new child();
c1.sound();
}
}

o/p Animal Sound is very louded.

Q. F./Q.No.

Q8.b

→ With lambda expression

@FunctionInterface
interface class_Name {
public void sound ();
}
public class Main {
public static void main (String args[])
{

class_Name s1 = new class_Name ();
s1. sound () -> {

System.out.print ("Animal sound
is very loaded");

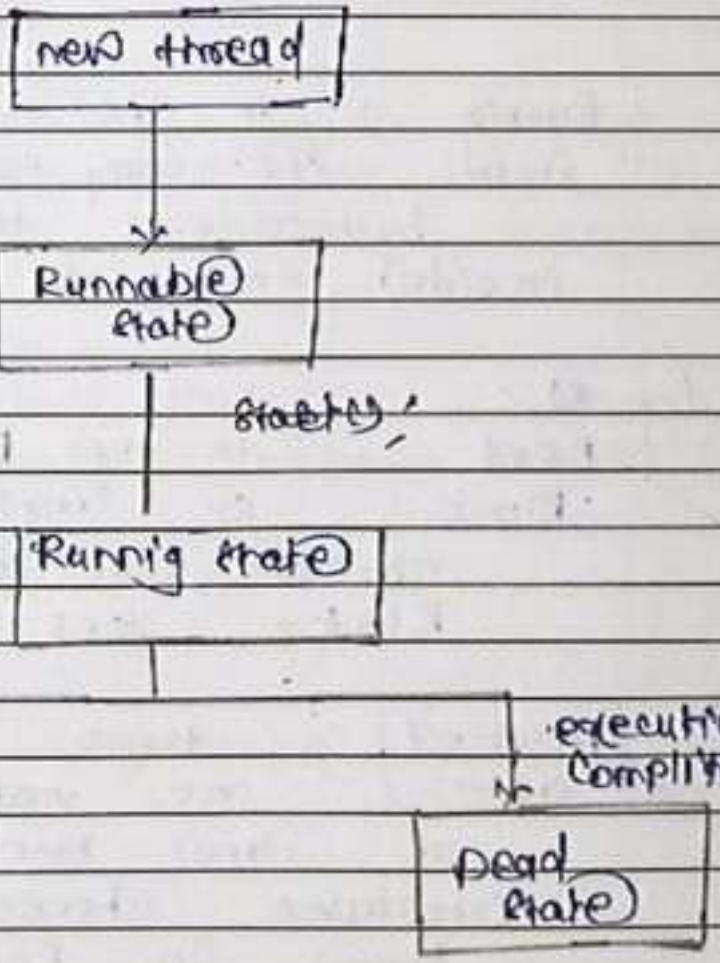
}

}

Output
// Animal. sound is very loaded.

Q9] Thread life cycle :- it is the process in which the created new thread passes through the states.

- the states are from compilation to the execution & till the thread terminates.
- this cycle shows from how many stage the thread passes while execution.



Q. K./Q.No.

Q 9

Q7

new thread state → When the object is created of the thread class at that time the thread goes inside this new thread state

Runnable state → the create thread waits in the Runnable state till the start method calls. & JVM Scheduler choosed the thread to be Run.

Running state → In the state of running the execution of the bytecode that has come inside this block is started.

Dead Block → In this Block the execution is completed & the thread that has reache this Blocke does not restart again.

Blocked state → there are multiple thread are ready to execute at time but the java scheduler choosed any one at time to Remain thread wait till the choosed thread completes execution.

Q.9

b] exception :- the exception is an unexpected error that can be appear at the execution time because of some logical & memomory allocation

Resone

public class

- Checked exception :- this is the unexpected exception that has appear & the programe hasn't written the logic to overcome that exception while apear

- In the program of checke exception the execution does not stop. It continoue even if any exception appears

```
public class Main {
public static void main(String args[]){
    int a = 10;
    int b = 0;

int Result = a/b;    // Arethimatic exception.
  System.out.print(result)
```

Q.No.

Q9

b→

```
public class Main {
public static void main (String args[])
    { int a=10, int b=0;
    try {
    int result = a/b;
    System.out.println(result);
        }
Catch ( ArithimaticException e)
    {
System.out.print("there is an
                Arithmentic exception");
                +e);
    }
```

Checked exception

O/P = Arithmetic exception.

∴ here the try statement checks the code if any exception apears it does NOT stop execution it passes the control of execution to the Catch statement.

Q. No.

Q9

b] the Catch Statement takes the argument of the exception & also execute.

Unchecked exception

→ In this Method the Function or Method Can stop to execution at the time any logical error, file error, array out of Bond exception appear.

→ It will stop execution at that instance

```
public class Main {
public static void main(String args[])
  {
    int a = 0;
    int b = 10;

int result = b/a;   // ArithmeticException
System.
int c = 10;
int d = 5;
int result2 = c/d;
System.out.println(result2);
```

O/P ArithmeticException ....

	Q.No.						TOTAL
E							
M							

Q. K./Q.No.

Q 9]

b] → In the unchecked method the program execution will stop at the point the error appears.

- it does not executes remaining correct logic.

using checked method

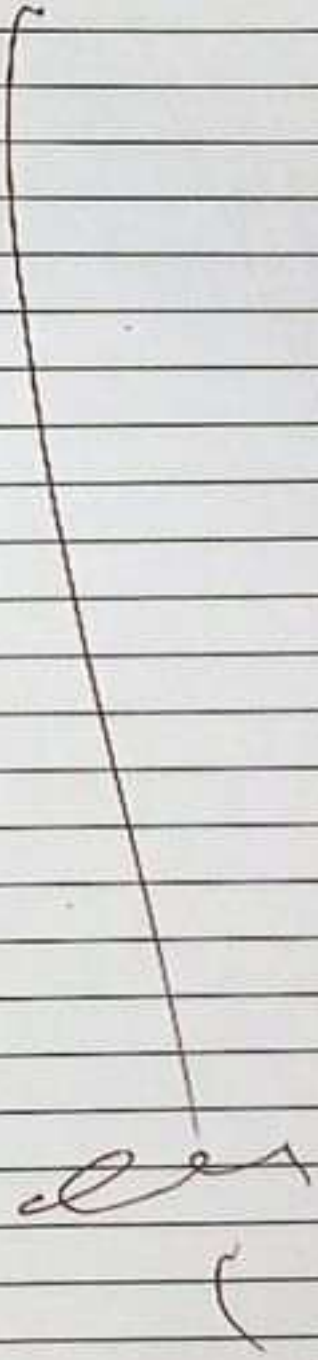
```
public class Main {
public static void main (String args [])
{
    a=10;
try {
    int a = 0;
    int b = 10;
    int result1 = a/b;
    int c = 10;
    int d = 5;
    int result2 = c/d;
    }
Catch ( ArithmeticException e)
    {
    System.out.print ("e");
    }
Finally {
        System.out.print (result1+e);
        }
```

Y. K. /Q.No.

Q9

b→ O/P Arithmetic Exception

2



	Q.No.						TOTAL
E							
M							

प्र. क्र./Q.No.